

Laboratorio 3 - Data Science

Pivi Riccardo

Novembre 2025

1 Esercizio 1 - Predirre l'ampiezza della banda proibita in composti binari

1.1 Materials Project

Il Materials Project (MP) è un ampio database computazionale che raccoglie proprietà strutturali ed elettroniche di materiali. Fornisce dati standardizzati e facilmente interrogabili tramite una API, utili per analisi e modellazione predittiva. L'obiettivo di questa prima parte dell'esperienza è allenare un modello per predirre il band gap (ampiezza della banda proibita). Si riporta il codice per prelevare dalla banca dati di MP 10 features di composti binari:

```
1 docs = []
2 with MPRester(api_key) as mpr:
3     # filtro: solo binari (nelements=2). Otteniamo tutti i doc summary con i campi richiesti.
4     docs = mpr.materials.summary.search(nelements=2, fields=fields)
5
6 print(f"Docs fetched: {len(docs)}")
7
8 X = []
9 y = [] #il nostro target l'energy gap
10 for d in docs:
11     features = np.array([
12         d.formation_energy_per_atom or 0.0,
13         d.energy_above_hull or 0.0,
14         d.density or 0.0,
15         d.volume or 0.0,
16         d.nsites or 0.0,
17         d.symmetry.number if d.symmetry is not None else 0.0,
18         d.efermi or 0.0,
19         d.total_magnetization or 0.0,
20         d.total_magnetization_normalized_vol or 0.0,
21         d.num_magnetic_sites or 0.0
22     ])
23     X.append(features)
24     y.append(d.band_gap) # band gap come target
25
26 X = np.array(X)
27 y = np.array(y)
```

dove i fields richiesti sono:

```
1 fields = [
2     "material_id",
3     "formula_pretty",
4     "band_gap", # fino a qui non usare come feature
5     "formation_energy_per_atom",
6     "energy_above_hull",
7     "density",
8     "volume",
9     "nsites",
10    "symmetry",
11    "efermi",
12    "total_magnetization",
13    "total_magnetization_normalized_vol",
14    "num_magnetic_sites"
15 ]
```

1.2 Correlazione

Si riporta il calcolo della correlazione tra le features e il band gap:

```
1 #correlazione
2 corr = np.zeros(10)
3 for j in range(10):
4     corr[j] = np.corrcoef(X[:, j], y)[0, 1]
```

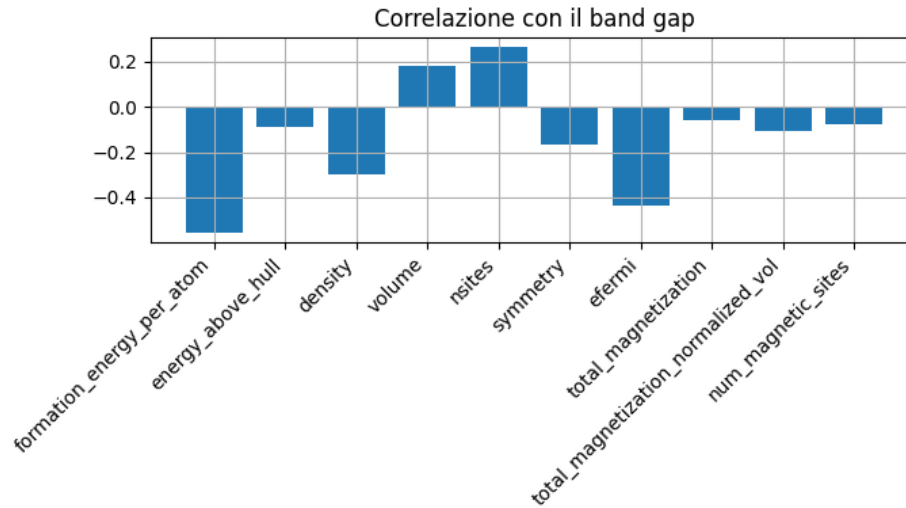


Figura 1: Grafico della correlazione fra band gap e le varie features.

I risultati numerici:

Feature	Correlazione
formation_energy_per_atom	-0.556
energy_above_hull	-0.089
density	-0.298
volume	0.180
nsites	0.265
symmetry	-0.170
efermi	-0.440
total_magnetization	-0.057
total_magnetization_normalized_vol	-0.105
num_magnetic_sites	-0.078

Tabella 1: Correlazione tra ciascuna feature e il band gap.

Notiamo come la correlazione maggiore è presente per l'energia di formazione per atomo e l'energia di fermi.

1.3 Regressione Lineare

Suddivisione in train e test e utilizzo di standard scaler:

```

1 X_train, X_test, y_train, y_test = train_test_split(
2   X, y, test_size=0.2, random_state=0
3 )
4
5 scaler = StandardScaler()
6 X_train_s = scaler.fit_transform(X_train)
7 X_test_s = scaler.transform(X_test)

```

Implementazione della regressione lineare:

```

1 lr = LinearRegression()
2 lr.fit(X_train_s, y_train)
3 y_pred_lin = lr.predict(X_test_s)
4
5 mse = mean_squared_error(y_test, y_pred_lin)
6 rmse_lin = np.sqrt(mse)

```

Si mostra il residuo e la predizione del modello lineare:

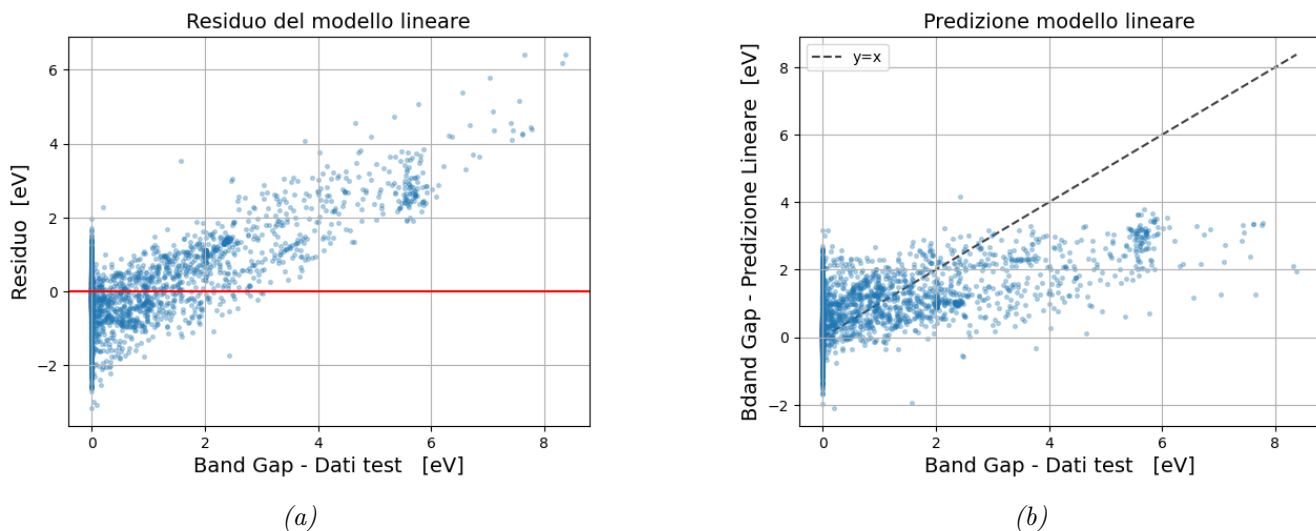


Figura 2: Grafico (a) mostra i residui del modello lineare, mentre (b) mostra il confronto fra la predizione del modello e il dato di test.

Si può vedere come il modello commetta un maggiore errore a predire energie di gap alte: il modello commette una sottostima per tali valori.

1.4 Regularizzazione con Ridge e Lasso

Si implementa nuovamente un modello lineare, ma con due tipi diversi di regularizzazione, Ridge e Lasso, studiando prima di tutto le performance del modello regularizzato al variare del parametro di regularizzazione:

```

1 alphas = np.logspace(-3, 8, 100)
2 ridge=linear_model.Ridge()
3 lasso = linear_model.Lasso()
4
5 coefs_ridge = []
6 coefs_lasso=[]
7
8 r2_train_ridge = []
9 r2_test_ridge = []
10
11 r2_train_lasso = []
12 r2_test_lasso = []
13
14 for a in alphas:
15     ridge.set_params(alpha=a)
16     ridge.fit(X_train_s, y_train)
17     coefs_ridge.append(ridge.coef_)

```

```

18
19 # Use the coefficient of determination R^2 as the performance of prediction.
20 r2_train_lasso.append(lasso.score(X_train_s, y_train))
21 r2_test_lasso.append(lasso.score(X_test_s, y_test))
22
23 lasso.set_params(alpha=a)
24 lasso.fit(X_train_s, y_train)
25 coeffs_lasso.append(lasso.coef_)
26
27 # Use the coefficient of determination R^2 as the performance of prediction.
28 r2_train_lasso.append(lasso.score(X_train_s, y_train))
29 r2_test_lasso.append(lasso.score(X_test_s, y_test))

```

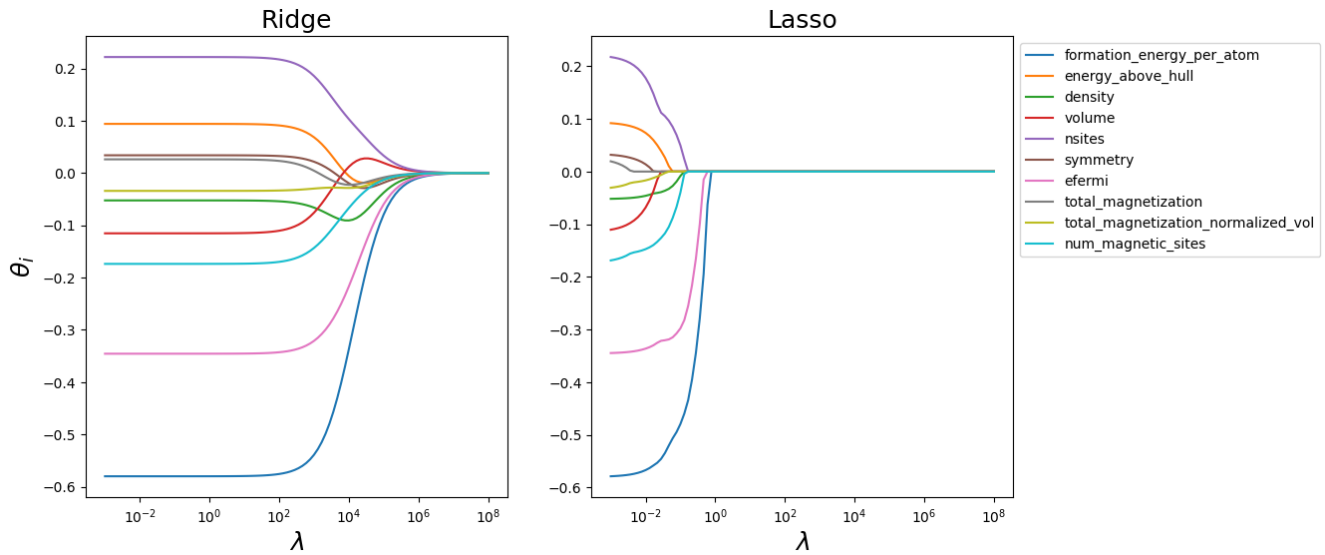


Figura 3: Grafico Coefficienti θ_i vs Parametro di regolarizzazione λ .

Si nota che i coefficienti in valore assoluto maggiori sono i coefficienti per l'energia di formazione per atomo e l'energia di fermi, ovvero le feature con maggiore correlazione.

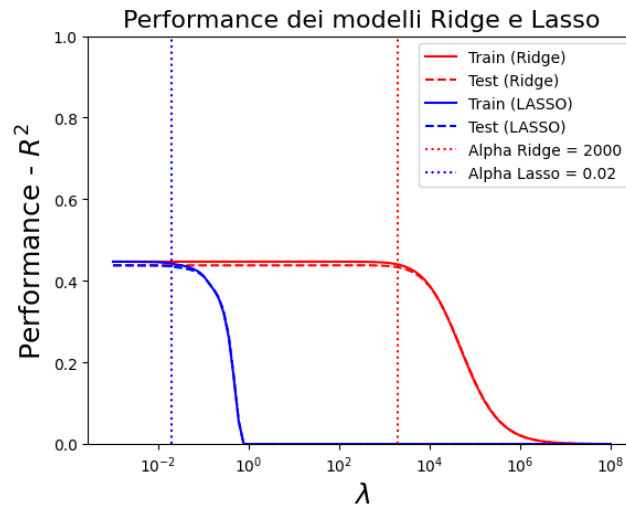


Figura 4: Grafico della performance dei due modelli sui dati di test e train, valutata con R^2 . Sono messi in evidenza - con rette verticali - i parametri di regolarizzazione usati successivamente.

Scelti due valori per il parametro di regolarizzazione si implementano i due modelli:

```

1 #Ridge
2 alpha_ride = 2000
3 # Predizione e calcolo metriche
4 ridge.set_params(alpha=alpha_ride )
5 ridge.fit(X_train_s, y_train)
6 y_pred_ride = ridge.predict(X_test_s)
7 r2_ride = r2_score(y_test, y_pred_ride)
8 rmse_ride = np.sqrt(mean_squared_error(y_test, y_pred_ride))
9
10 #Lasso
11 alpha_lasso = 0.02
12 # Predizione e calcolo metriche
13 lasso.set_params(alpha=alpha_lasso )
14 lasso.fit(X_train_s, y_train)
15 y_pred_lasso = lasso.predict(X_test_s)
16 r2_lasso = r2_score(y_test, y_pred_lasso)
17 rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))

```

1.5 Regressione Polinomiale

Si implementa un modello polinomiale di secondo grado:

```

1 degree = 2
2 poly = PolynomialFeatures(degree=degree, include_bias=True)
3
4 X_train_p = poly.fit_transform(X_train_s)
5 X_test_p = poly.transform(X_test_s)
6
7 lr_poly = LinearRegression().fit(X_train_p, y_train)
8 y_pred_poly = lr_poly.predict(X_test_p)
9
10 rmse_poly = np.sqrt(mean_squared_error(y_test, y_pred_poly))

```

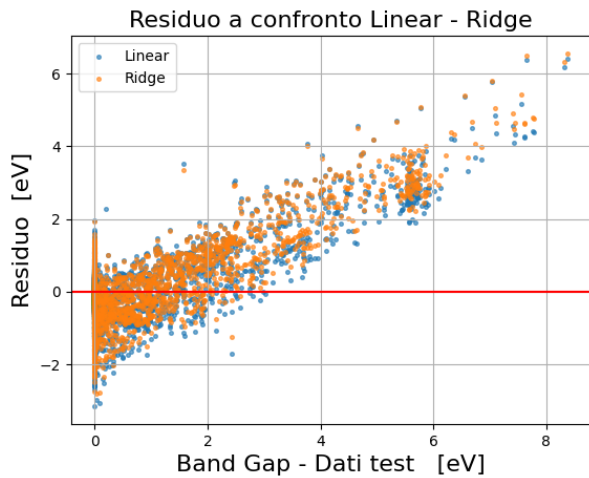
1.6 Valutazioni finali

Si riportano l'RMSE e R^2 per ogni modello:

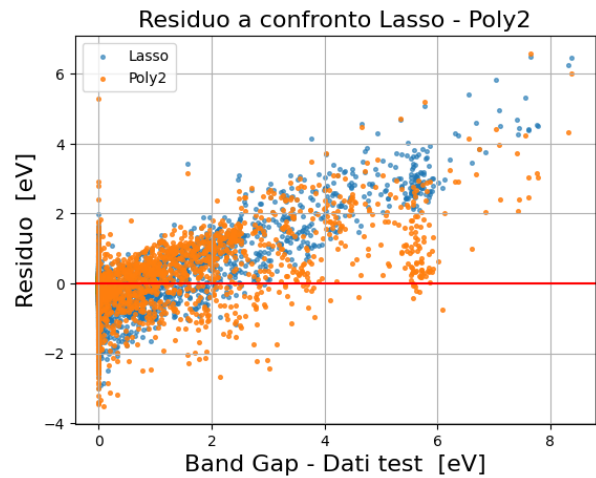
Modello	RMSE	R^2
Linear	0.8499	0.4379
Ridge	0.8534	0.4332
Lasso	0.8521	0.4349
Poly (deg=2)	0.6886	0.6309

Tabella 2: Confronto dei modelli: performance in termini di RMSE e R^2 .

Si riportano i grafici per il confronto fra i residui dei vari modelli:



(a)



(b)

Figura 5: Grafico (a) mostra il confronto dei residue fra modello lineare e il modello con la regolarizzazione Ridge. Il (b) mostra lo stesso confronto fra regolarizzazione Lasso e il modello polinomiale di grado 2.

In conclusione la regolarizzazione non ha migliorato la capacità predittiva del modello, mentre il modello polinomiale del secondo grado riesce a predire meglio i band gap. La regolarizzazione Ridge e Lasso non migliorano le prestazioni perché il modello lineare soffre principalmente di capacità predittiva insufficiente e non di overfitting. Penalizzare ulteriormente i coefficienti riduce ancora la flessibilità del modello, degradando leggermente la performance. Il miglioramento ottenuto con il modello polinomiale di grado 2 conferma che il problema non era l'eccesso di complessità, ma la sua insufficienza.

2 Regressione Logistica e metodo PCA per la banca dati MNIST

2.1 MNIST

La banca dati MNIST contiene 70.000 immagini in scala di grigi di cifre scritte a mano, ciascuna normalizzata in un formato 28×28 pixel. È uno dei dataset di riferimento nella data science per testare e confrontare algoritmi di classificazione e tecniche di riduzione della dimensionalità.

```
1 X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
2 y = y.astype(int)
```

2.2 Train e Test

I dati sono stati suddivisi in Train e Test e sono stati riscalati utilizzando Standard Scaler:

```
1 X, y = shuffle(X, y, random_state=42)
2 # 50.000 per training
3 X_train, y_train = X[:50000], y[:50000]
4
5 # 10.000 per test
6 X_test, y_test = X[50000:60000], y[50000:60000]
7
8 print("Train set:", X_train.shape)
9 print("Test set:", X_test.shape)
```

```

11 scaler = StandardScaler()
12 X_train_scaled= scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)
14

```

2.3 Regressione Logistica

Il modello allenato per riconoscere le cifre è un modello di regressione logistica:

```

1 model = LogisticRegression(max_iter=1000, solver='saga', tol=0.1, verbose=1)
2 model.fit(X_train_scaled, y_train)

```

Per valutare il modello, si sono calcolate l'accuratezza e la sparsità:

```

1 #accuratezza
2 y_pred = model.predict(X_test_scaled)
3 print('Accuracy:', accuracy_score(y_test, y_pred))
4 #sparsita'
5 sparsity = (np.sum(model.coef_==0)/model.coef_.size) *100
6 print(f"Sparsity: {sparsity}")

```

I risultati sono: Accuracy: 0.8856 Sparsity: 10.20

I coefficienti del modello:

```

1 weights = np.copy(model.coef_)
2 scale = np.abs(weights).max()
3 print(scale)

```

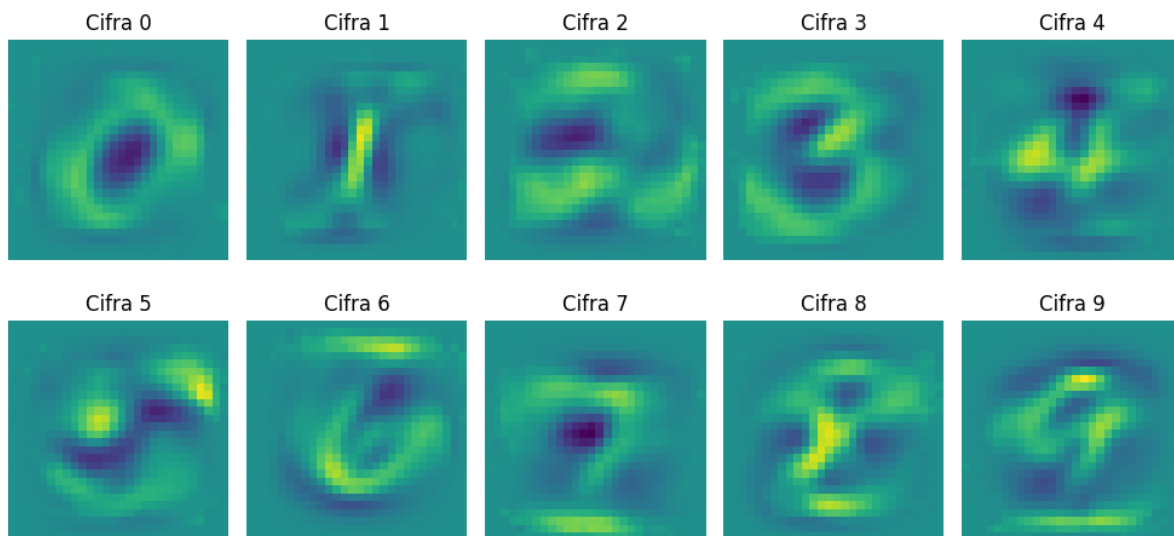


Figura 6: Rappresentazione grafica dei pesi imprati dal modello.

2.4 PCA e ricostruzione di un immagine

```

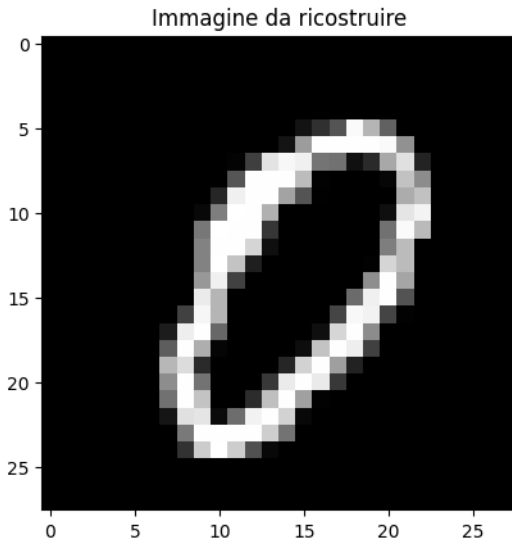
1 img_idx = 22#indice dell'immagine che ricostruisco
2
3 X_example = X.iloc[img_idx].to_numpy().reshape(28, 28) #serve una matrice 28x28 e il dato in panda
4
5 plt.imshow(X_example, cmap='gray')
6 plt.title("Immagine da ricostruire")
7 plt.show()
8
9 #PCA con n componenti per ricreare l'immagine
10 n_components = 100
11 pca = PCA(n_components=n_components)
12 print(pca)
13
14 X_pca = pca.fit_transform(X)

```

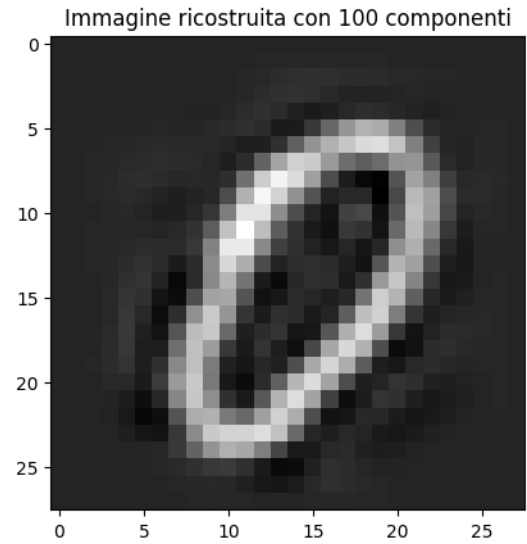
```

15
16 # Ricostruire l'immagine usando solo le prime 50 componenti
17 X_reconstructed = pca.inverse_transform(X_pca)
18 reconstructed_image = X_reconstructed[img_idx].reshape(28, 28)
19
20 plt.imshow(reconstructed_image, cmap='gray')
21 plt.title(f"Immagine ricostruita con {n_components} componenti")
22 plt.show()

```



(a)



(b)

Figura 7: (a) 22-esima immagine del data set. (b) ricostruzione utilizzando la PCA con 100 componenti della 22-esima immagine del dataset.

2.5 Varianza spiegata e Proiezione in 2 dimensioni con PCA

```

1 pca = PCA()
2 X_pca = pca.fit_transform(X)
3
4 # Varianza spiegata cumulativa
5 explained_variance_cumulative = np.cumsum(pca.explained_variance_ratio_)

```

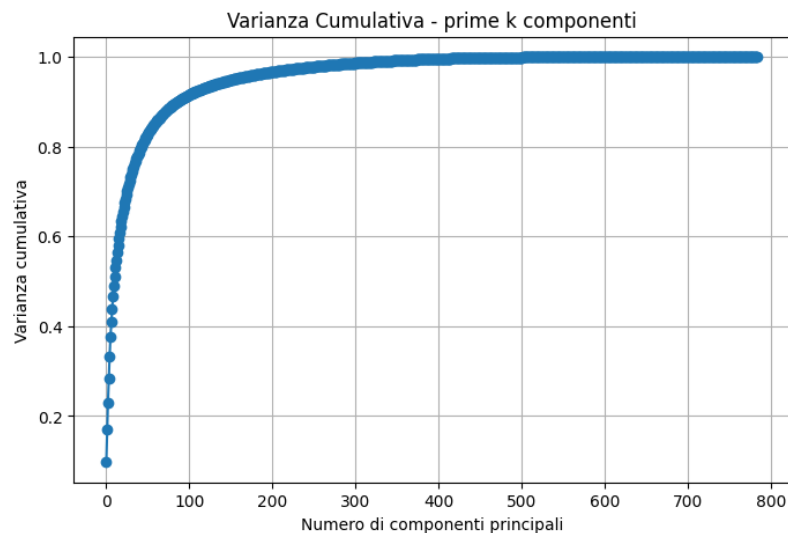


Figura 8: Grafico della varianza cumulativa.


```

1 pca_2d = PCA(n_components=2)
2 X_pca_2d = pca_2d.fit_transform(X)

```

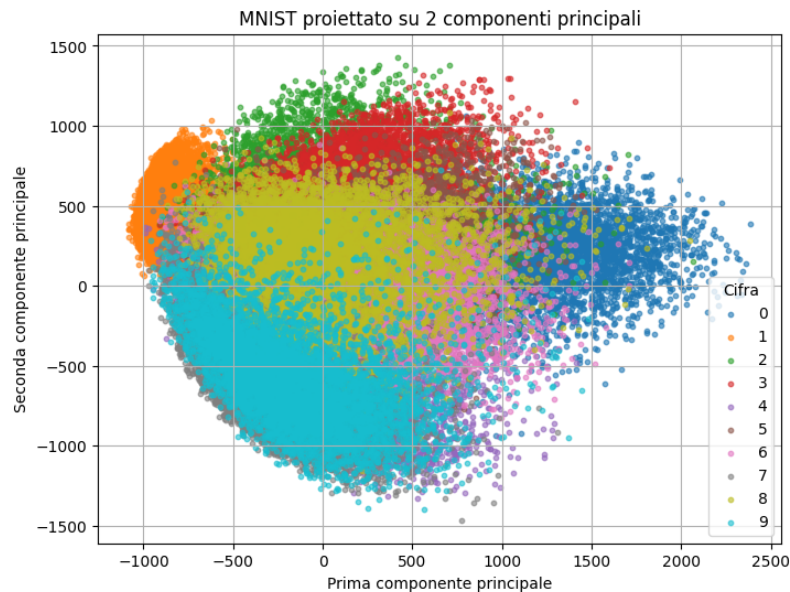


Figura 9: Proiezione del dataset su 2 dimensioni utilizzando la PCA.

La PCA è un algoritmo lineare e in questo caso la proiezione non permette di vedere la struttura del data-set e quindi non è possibile distinguere agglomerati di dati. Un altro algoritmo di riduzione di dimensionalità come t-SNE può riuscirci.

2.6 EXTRA: t-SNE

```

1 from sklearn.manifold import TSNE
2
3 pca_50= PCA(n_components=50) # riduco prima a 50 dimensioni per velocizzare t-SNE
4 X_pca_50 = pca_50.fit_transform(X)
5
6 tsne_2d = TSNE(
7     n_components=2,
8     perplexity=30,
9     max_iter=500,
10    method="barnes_hut",
11    random_state=42
12 )
13
14 X_tsne_2d = tsne_2d.fit_transform(X_pca_50)

```

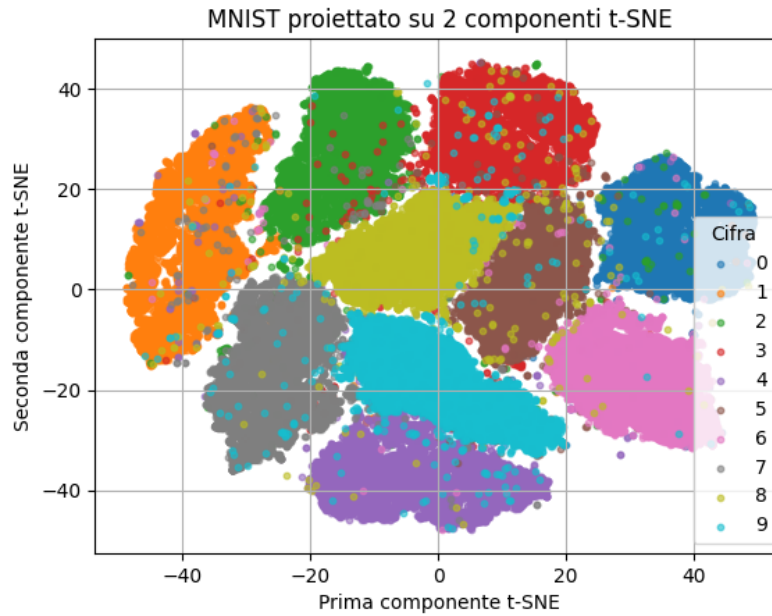


Figura 10: Proiezione del dataset su 2 dimensioni utilizzando la t-SNE.

3 Auto-facce e algoritmo PCA

3.1 allFaces.mat

Gli eigenfaces (autofacce) sono autovettori estratti da un insieme di immagini facciali mediante PCA. Questa ci permette di descrivere le immagini in un nuovo spazio di coordinate, permettendo di ricostruire altre immagini usando questa nuova base di vettori. Il data-set utilizzato è allFaces.mat, si riporta il codice per prelevare i dati:

```
1 mat_contents = scipy.io.loadmat(os.path.join('.', 'allFaces.mat')) #legge un file di matlab e lo carica in un dizionario python
2 print(mat_contents.keys())
3 faces = mat_contents['faces'] #matrice delle immagini delle facce: 2410 facce da 32256 pixel
4 n = int(mat_contents['n'].item()) #altezza immagine
5 m = int(mat_contents['m'].item()) #larghezza immagine
6 nfaces = np.ndarray.flatten(mat_contents['nfaces'])
7 print(f"righe: {n}, colonne: {m}")
8 print(faces.shape) #(n_pixel, n_facce)
```

Il dataset è composto da 2410 immagini da 32256 pixel (192. Gli individui ritratti nelle foto sono 38, ognuno con più foto (circa 60) con diverse illuminazioni.

3.2 PCA ed Autofacce

```
1 n_face = np.sum(nfaces[:38]) #utilizzo tutte le facce (38 persone)
2
3 trainingFaces = faces[:, :n_face]
4 avgFace = np.mean(trainingFaces, axis=1)
5
6 # Trovo le autofacce sui dati mean-subtracted
7 X = trainingFaces - np.tile(avgFace, (trainingFaces.shape[1], 1)).T
8 U, S, VT = np.linalg.svd(X, full_matrices=0)
```

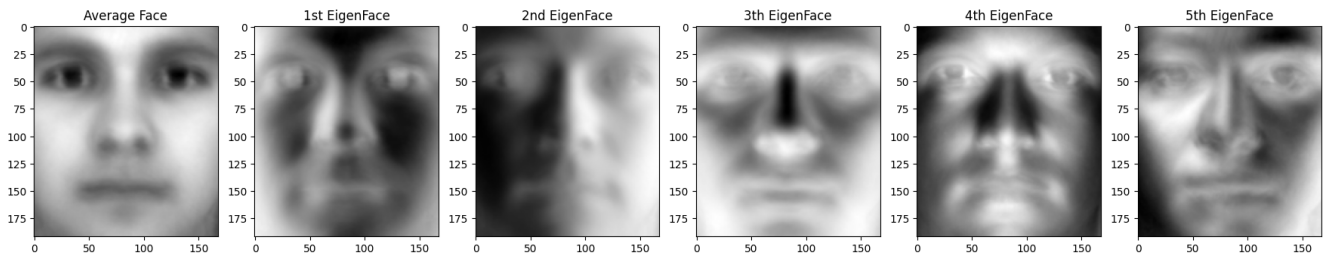


Figura 11: Immagine che mostra partendo da sinistra: la faccia media, la prima autofaccia, la seconda, la terza, la quarta e la quinta.

Le autofacce sono gli autovettori che forniscono una nuova base ordinata in base alla varianza dei dati su quel raggio dello spazio 32256. E' possibile interpretare qualitativamente le autofacce: ognuna di esse mostra una diversa illuminazione del viso e lineamenti maggiormente marcati; ad esempio la prima autofaccia descrive i lineamenti fondamentali di un viso ed un'illuminazione frontale, la seconda un'illuminazione laterale, mentre la quinta mette in forte contrasto le due sopracciglia.

3.3 Ricostruzione immagine di una persona

Utilizzando la PCA si possono ricostruire foto anche non presenti nel dataset, per esempio il viso di Fantozzi e Marilyn Monroe. Inoltre, si può controllare come la ricostruzione diventi più accurata man mano che aumenta il numero r di componenti principali che utilizziamo. Per farlo è necessario caricare l'immagine e renderla compatibile col dataset:

```
1 img = mpimg.imread('Marilyn.jpg')
2 img_gray = rgb2gray(img)
3
4 # resize correttamente
5 X = resize(img_gray, (n, m), anti_aliasing=True)
6
7 # match intensity scale
8 X = X * 255.0
9
10 # flatten in column-major, come 'faces'
11 img_vector = X.T.flatten()
```

Poi si effettua la ricostruzione e si mostrano i risultati al variare di r :

```
1 testFaceMS = img_vector - avgFace
2
3 testFaceMS = img_vector - avgFace
4 r_list = [25, 100, 400, 800, 1600, n_face]
5
6 mse_history_mari = []
7
8 fig, axes = plt.subplots(1, len(r_list)+1, figsize=(18, 5)) # +1 per originale
9
10 # Plotto prima l'immagine originale
11 axes[0].imshow(np.reshape(img_vector, (m, n)).T, cmap='gray')
12 axes[0].set_title('Originale')
13 axes[0].axis('off')
14
15 # Poi ricostruzioni
16 for i, r in enumerate(r_list):
17     coeff = U[:, :r].T @ testFaceMS
18     reconFace = avgFace + U[:, :r] @ coeff
19
20     mse = np.mean((img_vector - reconFace)**2)
21     mse_history_mari.append(mse)
22
23     axes[i+1].imshow(np.reshape(reconFace, (m, n)).T, cmap='gray')
24     axes[i+1].set_title('r_U=' + str(r))
25     axes[i+1].axis('off')
26
27 plt.tight_layout()
28 plt.show()
```

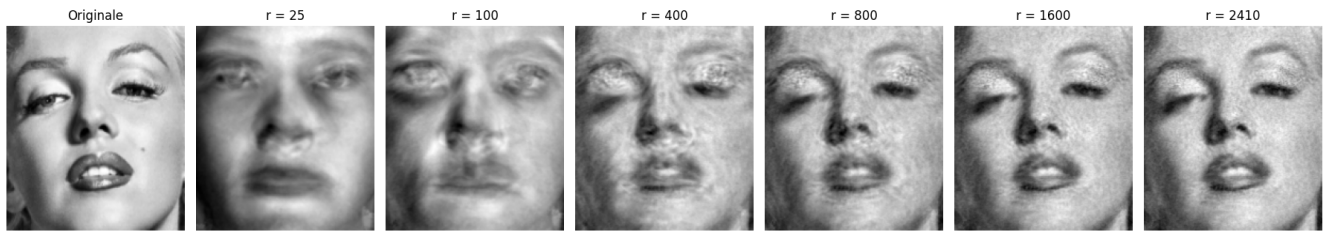


Figura 12: Ricostruzione del viso di Marilyn Monroe attraverso la PCA al variare di r . A sinistra la foto originale.

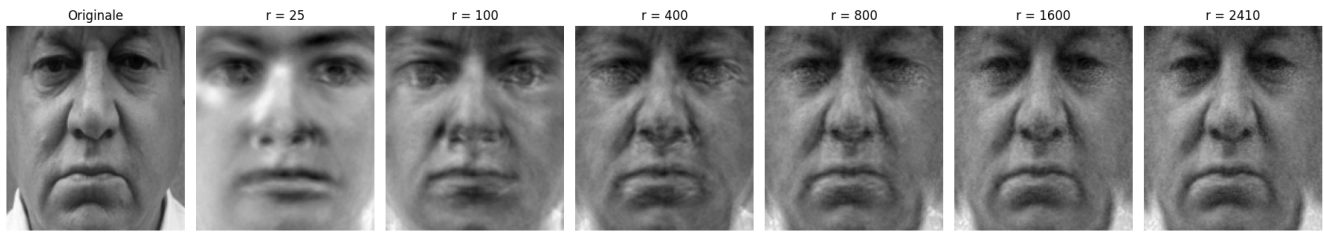


Figura 13: Ricostruzione del viso di Fantozzi attraverso la PCA al variare di r . A sinistra la foto originale.

I coefficienti usati in una ricostruzione sono stati calcolati:

```
1 coeff = U[:, :r].T @ testFaceMS
```

Riportiamo i coefficienti riportati per la ricostruzione della foto di Fantozzi:

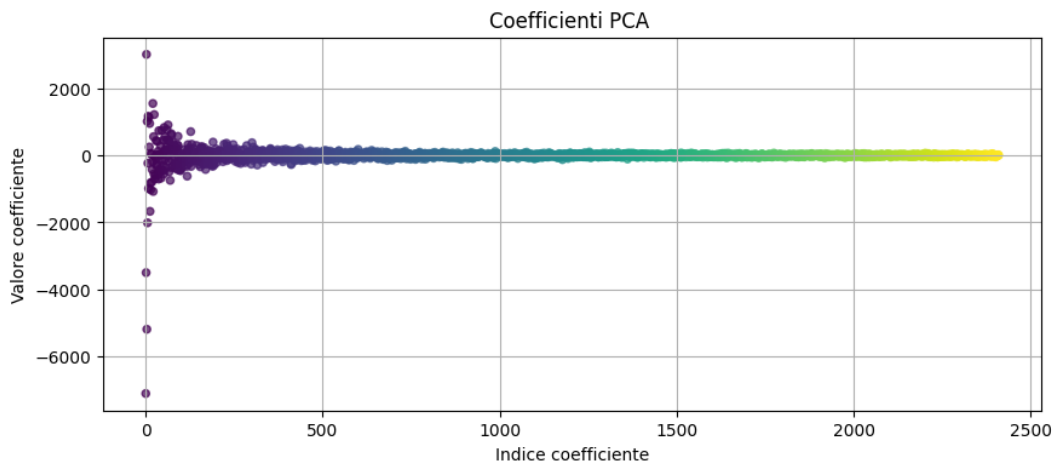


Figura 14: I coefficienti calcolati per la ricostruzione dell'immagine del viso di Fantozzi. Il colore è usato per enfatizzare il valore dell'indice del coefficiente.

3.4 Ricostruzione immagine di un oggetto

E' possibile svolgere la stessa ricostruzione per oggetti diversi da volti ed ovviamente non presenti nel dataset. Si riporta il risultato della ricostruzione di un tavolo da biliardo:

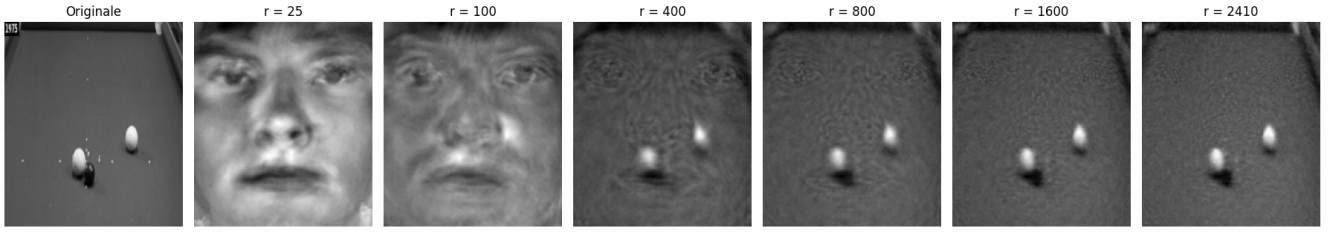


Figura 15: Ricostruzione dell'immagine di un tavolo da biliardo attraverso la PCA al variare di r . A sinistra la foto originale.

Risultato sorprendente che ci dimostra una robustezza qualitativa della base di autofacce trovata grazie al data-set.

3.5 Perdita di Informazione

La ricostruzione non è perfetta, è possibile però valutare la perdita di informazione tramite l'errore commesso nella ricostruzione stessa.

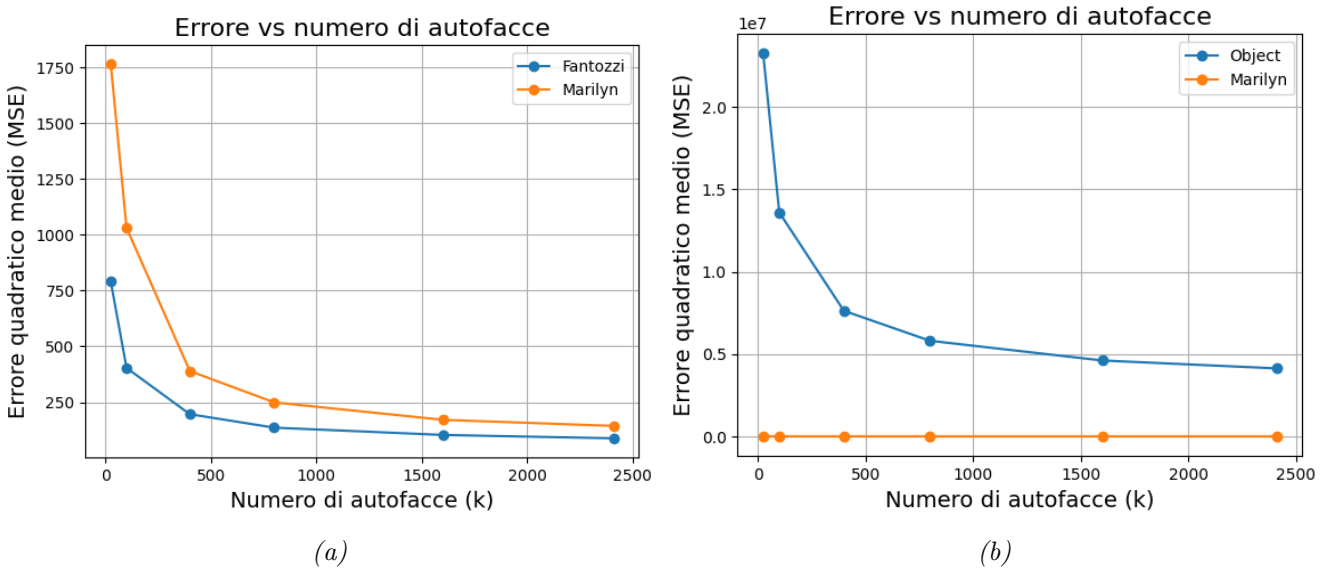


Figura 16: Grafici dell'errore quadratico medio al variare del numero k di autofacce usate per la ricostruzione. (a) Confronto ricostruzione Marilyn e Fantozzi. (b) Confronto ricostruzione oggetto (palle da biliardo) e Marilyn.